

Parallel Programing

By Josef Arango, Sage Cauley, Wesley Simms

Abstract

This paper explores how parallel and distributed computing techniques can be leveraged to accelerate a line-based text processing task. The work is carried out on Beocat, the High-Performance Computing (HPC) cluster at Kansas State University, using real-world data in the form of a large Wikipedia article. To implement parallelism, we use three standard approaches: Pthreads for shared memory threading, MPI for distributed memory communication, and OpenMP for directive-based parallelization. Our analysis focuses on three core performance metrics: execution time, CPU utilization, and memory usage across the various implementations. In addition, we examine page fault behavior to understand the memory access patterns and system-level performance differences. Through these experiments, this study provides insights into the trade-offs, efficiency, and scalability of different parallel programming models.

Introduction:

This project focuses on optimizing the computation for the max ASCII value in each line of a large text file (we were given a wiki article for each line). The aim is to gather data and perform an analysis over that data to see which way is the best and how they differ in resource use. They were implemented and evaluated using:

- MPI for distributed memory systems
- Pthreads for shared memory systems
- OpenMP for shared memory parallelism

We gathered the data using 3 different systems, the first being perf for CPU utilization, kstat for getting memory usage, and clock in the code to get the time it took to run.

Implementation:

Pthreads-

For our Pthread implementation we took in 2 arguments to our main function for running the program that is first the file to run and second how many cores to use for that specific run. From there it reads the files line by line and stores them dynamically into memory. After that it initializes threads and assigns them a portion of lines from the file to process. It will then wait for all the threads to get finished and then print out either the time takes or the max character of each line along with a line number. To finish it all we clean up all the memory and anything extra.

MPI-

For our MPI implementation we took in 1 argument to our main function which was the file to read from. It opens the files and reads the content line by line into memory and broadcasts the total number of lines to all other processes. Then it scatters portions of the lines to each process. Each process then will independently compute the max ASCII character for its assigned lines. The results are gathered back and then output into the terminal with the lines number and max ASCII.

OpenMP-

Takes in 2 arguments into main which the first is the file to read the second is the number of threads to run it with. Then it will read the text file line by line and use OpenMP to

parallelize the computation. After dynamically loading all the lines from the file into memory the program allocates an array to store the max ASCII character values then uses a parallel for loop to process each line. Execution time for the process is measured and printed. Finally, the program frees all memory to prevent leaks.

Evaluation:

Tools used-

We used three different tools to collect the data over the first one was clock where we implemented in our code collecting the current time to start then the current ending time to stop and subtracted and divided to get our overall time the program ran for. Then we used perf to gather data over the CPU utilization and how that differed across all the different methods and cores. Lastly, we used kstat to gather data over the memory usage and how that differed across all the different tests. Excel was used for creating our graphs over our data including a standard deviation. Then python was used for overall creation of data like our smaller file sizes and formatting our data to a csv file that was easier import into excel.

Pthreads-

When evaluating each of the threads we did the following tests found in table 1. Each of the tests were run a total of 5 times top get an average of the processing time. Each of these tests were done with a total memory of 7GB to have more than enough space for the files to be loaded into memory. All the tests were restricted to having only one thread per core and constrained to the Mole compute node.

Core configurations used:

1 Core	5 Cores	10 Cores	15 Cores	20 cores
--------	---------	----------	----------	----------

Core / line configurations used:

5 Cores with 1K lines	5 Cores with 10K lines	5 Cores With 100K lines	5 cores with 1M lines
-----------------------	------------------------	-------------------------	-----------------------

System / core configurations used:

10 Cores Between 2 Systems	20 Cores Between 2 Systems
----------------------------	----------------------------

Below you will find the first row of tests. When more cores were added the times drastically decreased the system time. As we got closer to 20 cores the time started to plateau. This is

due to the cost of creating a new thread that could have been used to actually run the program and taking valuable compute time from the system.

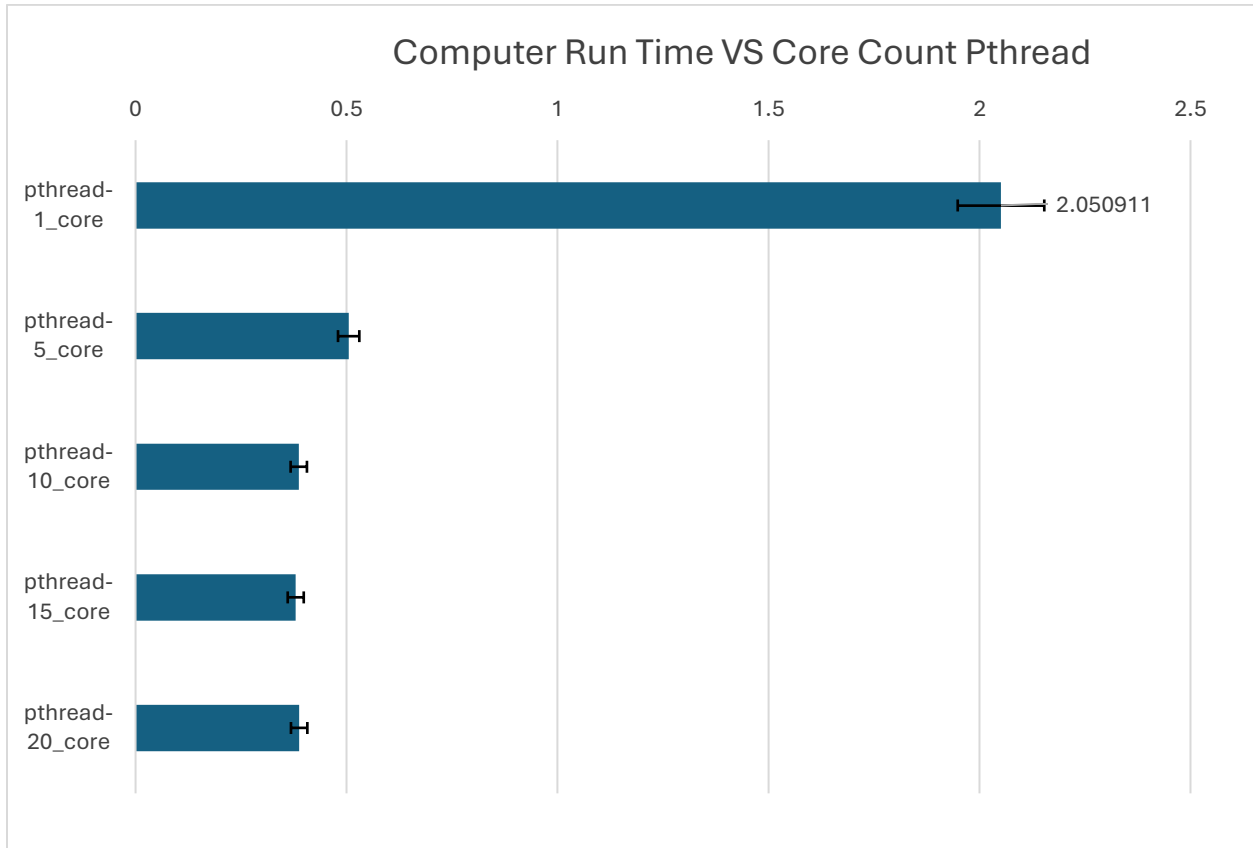


Figure 1: Computer Run Time VS The Core Count Used

The following graph shows the difference between 1 system and 2 system. All other variables were kept the same to limit the differences and get clean differences. Looking at the data it is noted that the 2 systems took longer than running on one system. This is to be expected as the process must transfer data between the two systems that will never be faster than on chip interconnect speeds. Note that 10 is somewhat close as it was somewhat reasonable to use two systems as the scheduler time was not significant enough to make a difference. Then with 20 cores it took over 2x the time caused by the information transfer.

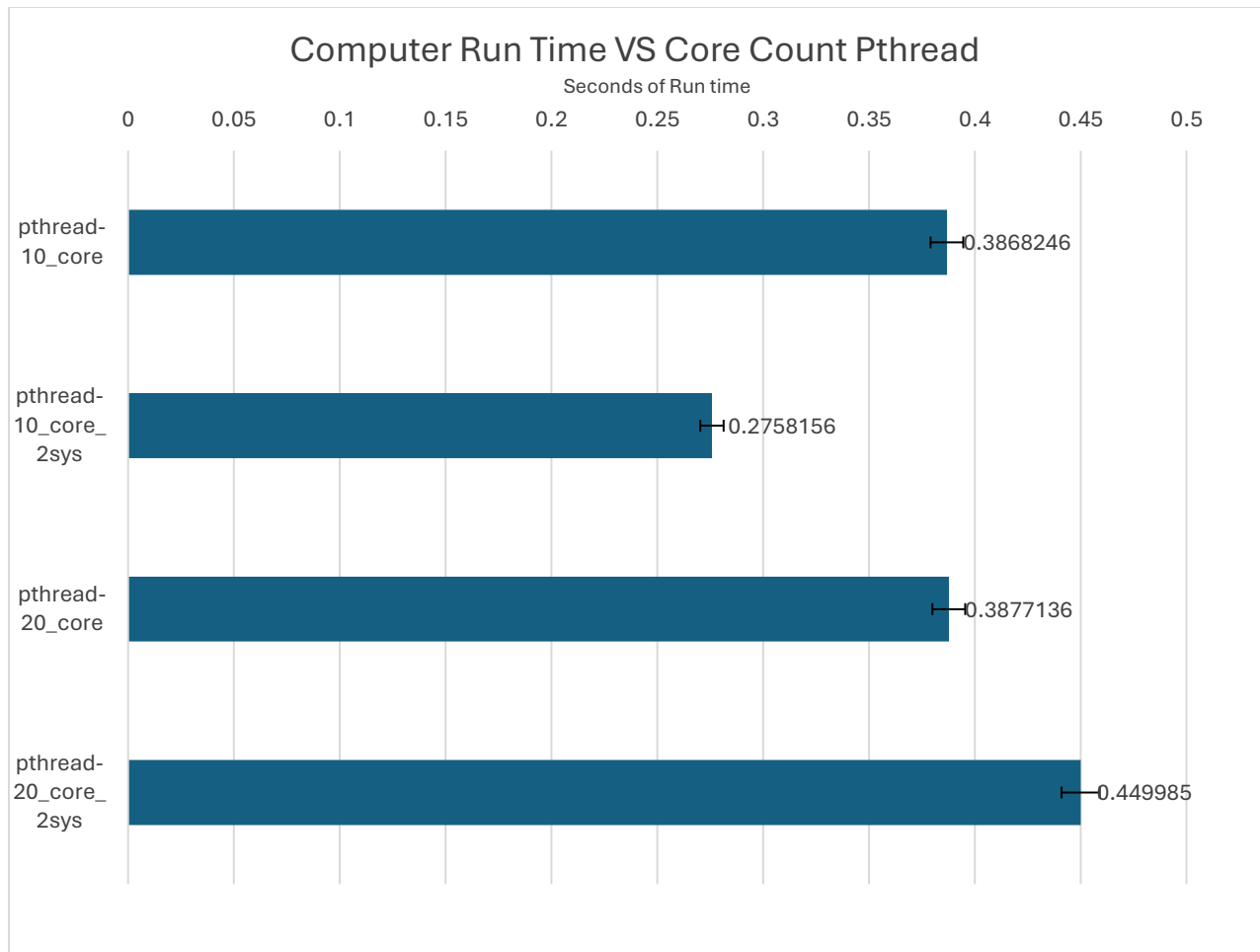


Figure 2: The Compute Run Time Between 1 and 2 systems

The following figure is the different file sizes. All these tests were done using 20 cores and the default configuration. Each of the files was built using a python script found in the appendix. The different sizes had the same number of elements in each line, but the number of lines were changed to 1k, 10k 100k and 1M lines. Then each of the 5 tests were run on each of the files. Looking at the graph it is evident that the number of the element's changes in the $O(n^2)$ in time.

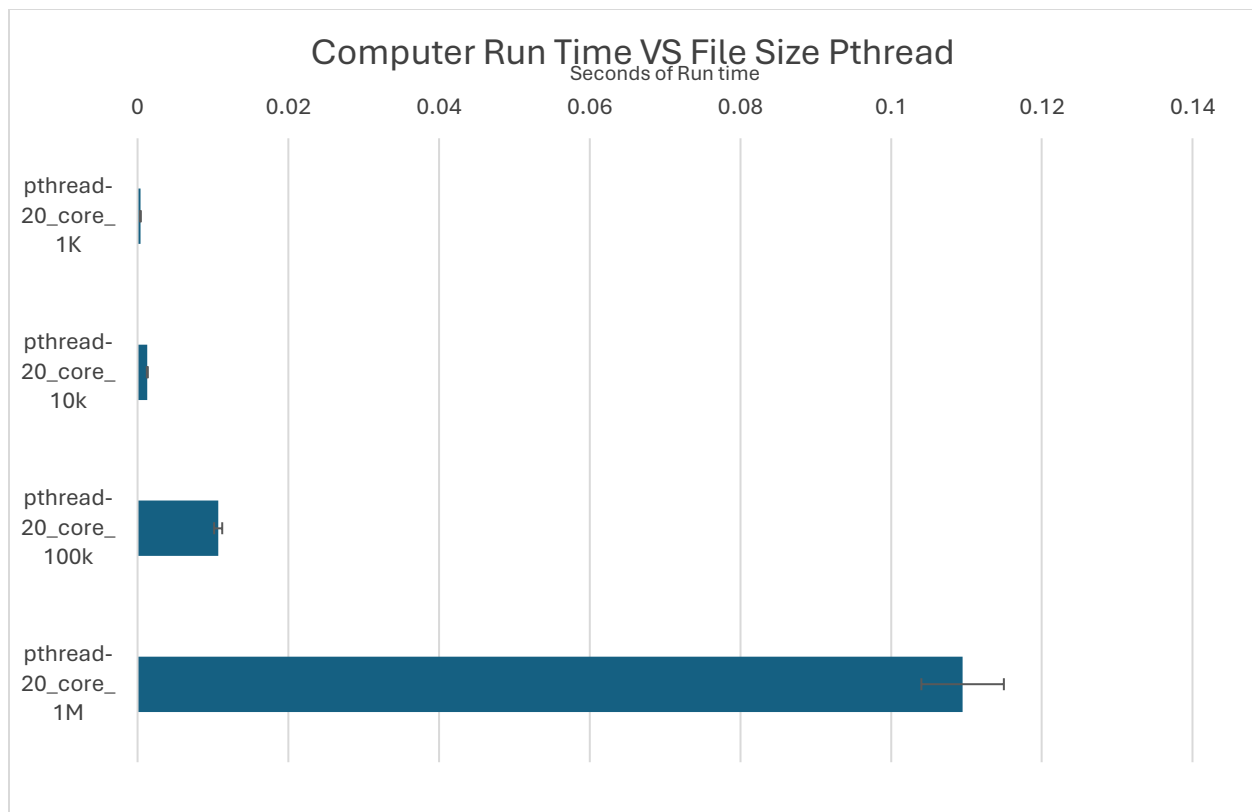


Figure 3: The Computer Run Time VS The Size of the File

Below is the over memory usage from kstat for pthread. Looking at them it is event that the programs seem to use no memory or under the amount that we would expect. The expected result would be the size of file to be read or the max amount that it could. Evidently it doesn't show up here. This could be due to many different reasons, mostly having to attribute to running on the computer node. An example of this is shown latter on with MPI.

21761552	1_core	mole053	1 n	1 c	0.00gb/	7gb	00:00:23	COMPLETED
21761553	5_core	mole054	1 n	5 c	0.00gb/	7gb	00:00:16	COMPLETED
21761554	10_core	mole059	1 n	10 c	0.00gb/	7gb	00:00:15	COMPLETED
21761555	15_core	mole058	1 n	15 c	0.00gb/	7gb	00:00:15	COMPLETED
21761556	20_core	mole055	1 n	20 c	0.00gb/	7gb	00:00:15	COMPLETED
21761557	10_core_2s	mole[053-054]	2 n	10 c	0.00gb/	14gb	00:00:16	COMPLETED
21761558	20_core_2s	mole[053-054]	2 n	20 c	0.00gb/	14gb	00:00:16	COMPLETED
21761559	5_core_1M	mole056	1 n	20 c	0.00gb/	7gb	00:00:01	COMPLETED
21761560	5_core_100	mole057	1 n	20 c	0.00gb/	7gb	00:00:01	COMPLETED
21761561	5_core_10K	mole060	1 n	20 c	0.00gb/	7gb	00:00:00	COMPLETED
21761562	5_core_1K	mole061	1 n	20 c	0.02gb/	7gb	00:00:01	COMPLETED

The following is the number of cores that were used with each of the systems. As more cores were available for usage but overall, the system utilization was not fully maxed out after 1 core. Its max utilization was 3.1244 with 20 cores. The computation was not as heavy on the system as to believe.

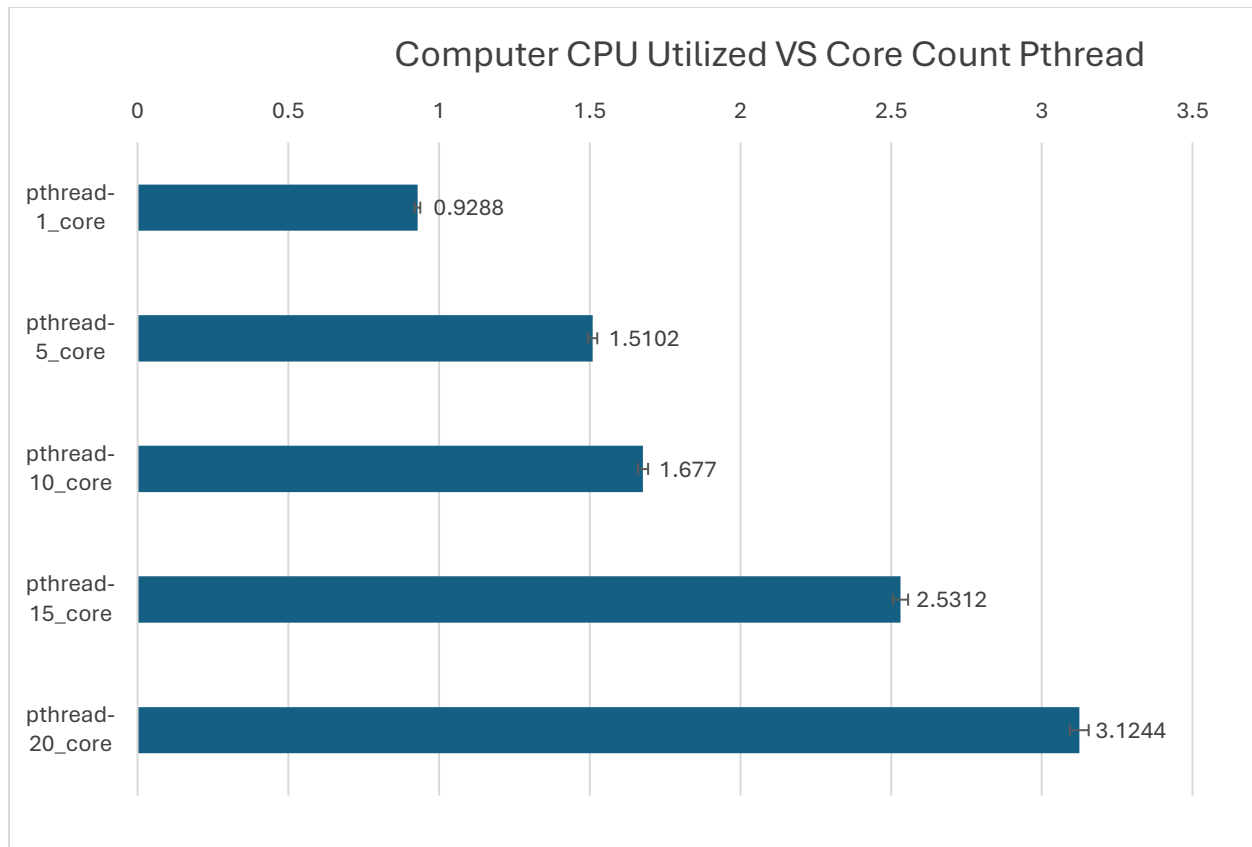


Figure 4: CPU utilization compared to core count

MPI-

When evaluating the effectiveness of the program it is evident that it did not do as well. This is due to the program using over double the lines than the other two programs. Looking over the code several times we were unable to find an issue. There were issues with figuring out how to distribute the chunks of memory between the processes. While threads share memory, MPI does not, which makes the logic for distributing differing length strings difficult. But working over this obstacle we were able to get the code to run in some form.

Below is the run time compared to the CPUs used where the overall look of the graph is correct. Despite having to process twice as many lines as the other program, it doesn't do too badly, running in almost double the time which would be somewhat expected for this issue.

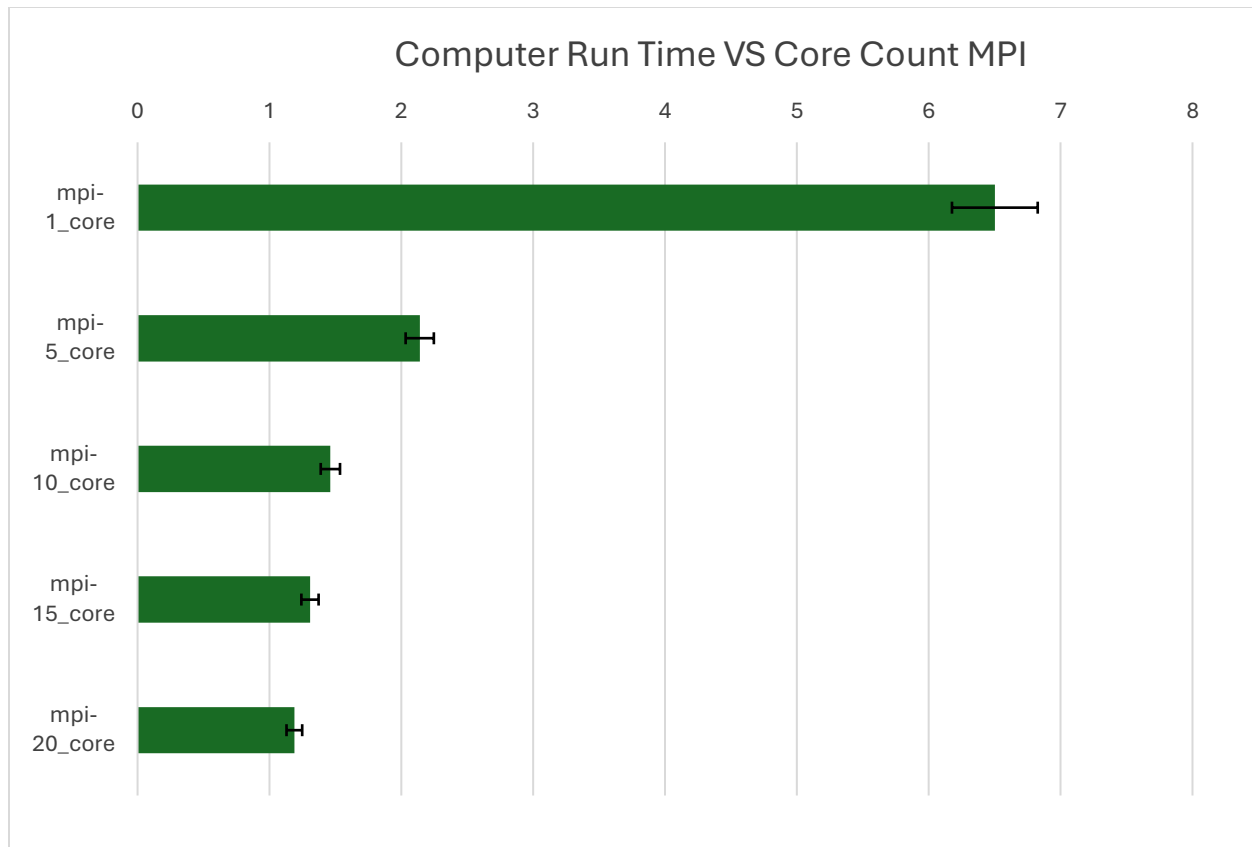


Figure 5: Computer Run Time VS The Core Count Used

Looking at the runtime with the systems, we can see the difference between 1 system and 2 systems. Similarly to our pthread implementation, we keep all the variables the same but only change the number of nodes. It should be noted that the 2 systems took longer than running on one system. This is to be expected as the process must transfer data between the two systems that will never be faster than on chip interconnect speeds. This also was exemplified by the system having to share the data between the two with double the amount of data.

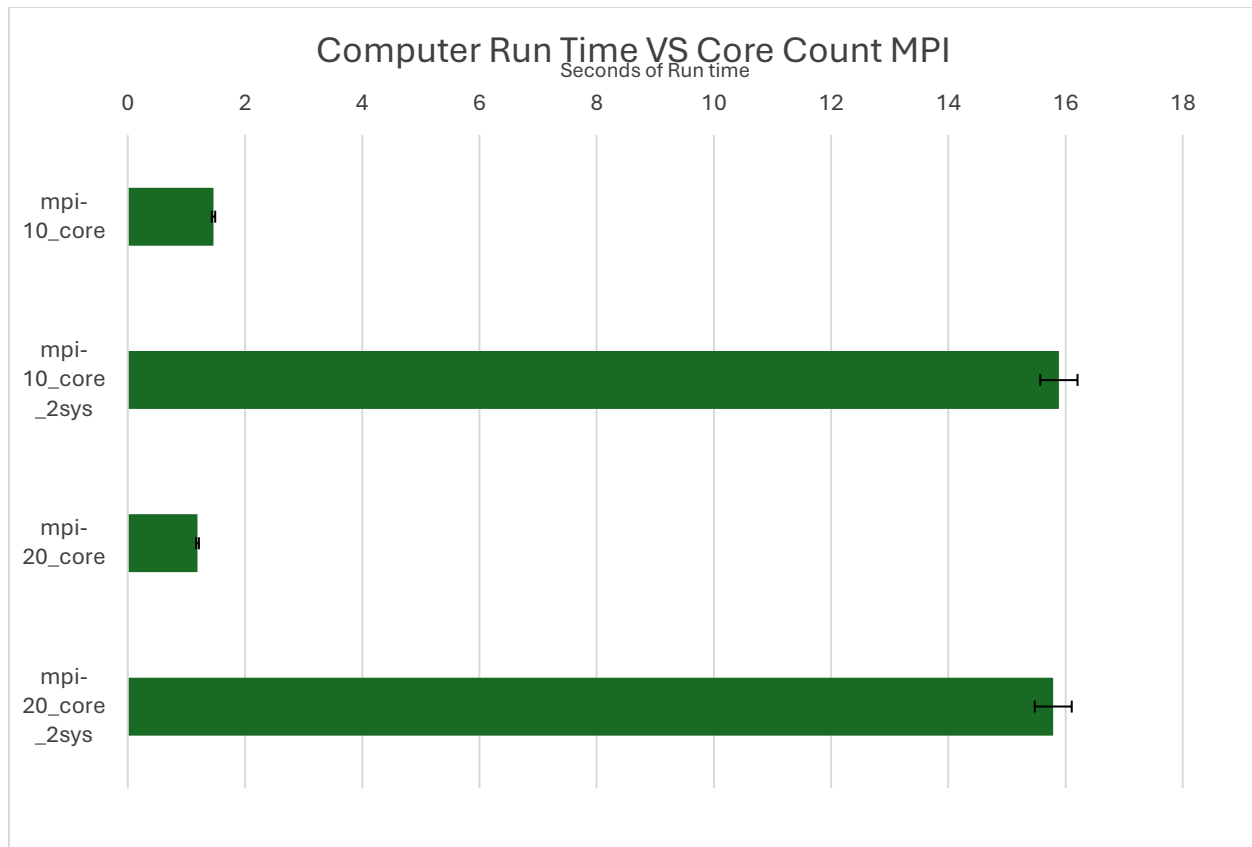


Figure 6: The Compute Run Time Between 1 and 2 systems

The following figure shows the different file sizes following the same trend as Pthread. All these tests were done using 20 cores and the default configuration. Each of the files was built using a python script found in the appendix. The different sizes had the same number of elements in each line, but the number of lines were changed to 1k, 10k 100k and 1M lines. Then each of the 5 tests were run on each of the files. Looking at the graph it is evident that the number of the element's changes in the $O(n^2)$ in time. Overall, it seems to align with expectations but with a lower time.

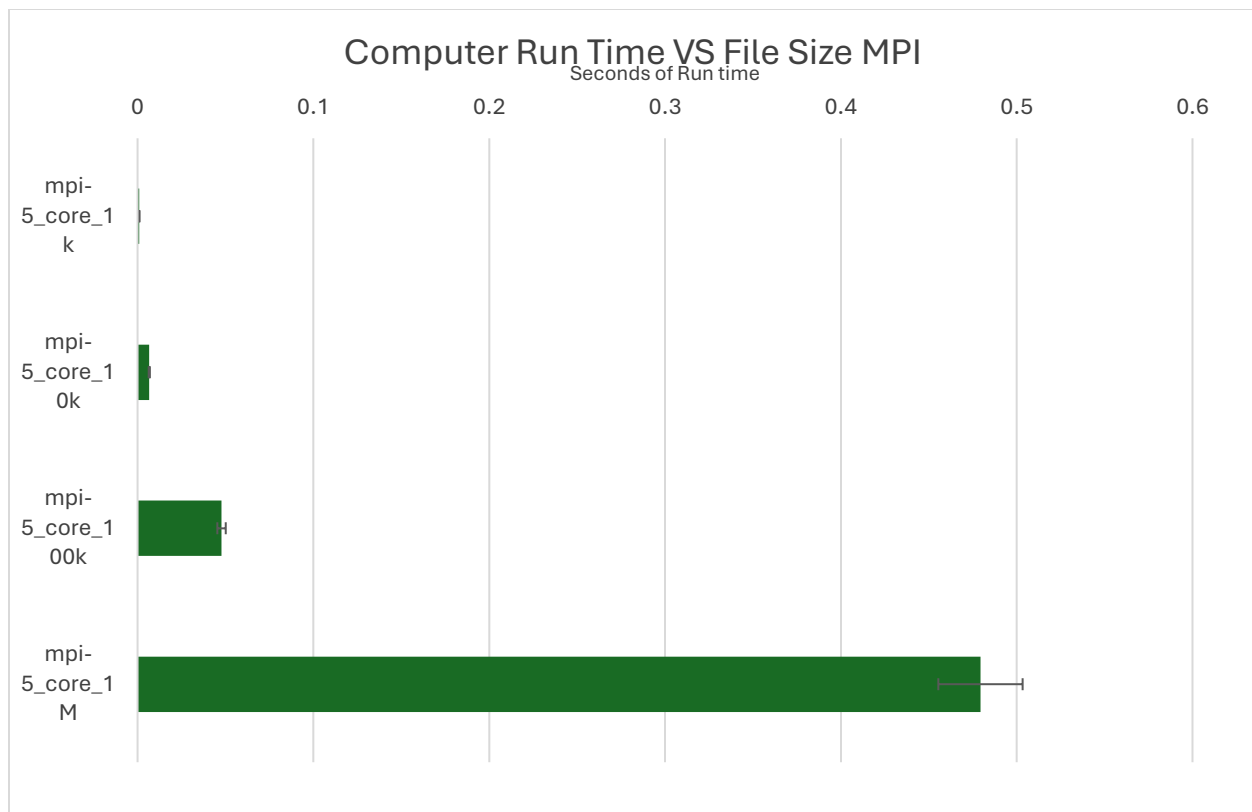


Figure 7: The Computer Run Time VS The Size of the File

For the memory usage we see following from the tests: It is evident that the 2 systems are using significantly more memory than most of the other processes. This is due to the latency of sending information between the two systems for synchronization. While the program for 1 core is an outlier in this situation with a usage of 3.49gb. This might be the system that is providing the memory for the other systems as it makes no sense for the other systems to not use any memory. Unless the system is putting the memory into cache

21761563	1_core	mole053	1 n	1 c	3.29gb/	7gb	00:00:46	COMPLETED
21761564	5_core	mole056	1 n	5 c	0.00gb/	7gb	00:00:25	COMPLETED
21761565	10_core	mole059	1 n	10 c	0.00gb/	7gb	00:00:21	COMPLETED
21761566	15_core	mole065	1 n	15 c	0.00gb/	7gb	00:00:21	COMPLETED
21761567	20_core	mole064	1 n	20 c	0.01gb/	7gb	00:00:20	COMPLETED
21761568	10_core_2s	mole[056-057]	2 n	10 c	2.74gb/	14gb	00:01:36	COMPLETED
21761569	20_core_2s	mole[056-057]	2 n	20 c	2.78gb/	14gb	00:01:36	COMPLETED
21761570	5_core_1M	mole060	1 n	20 c	0.00gb/	7gb	00:00:07	COMPLETED
21761571	5_core_100	mole066	1 n	20 c	0.00gb/	7gb	00:00:08	COMPLETED
21761572	5_core_10K	mole067	1 n	20 c	0.00gb/	7gb	00:00:08	COMPLETED
21761573	5_core_1K	mole068	1 n	20 c	0.00gb/	7gb	00:00:12	COMPLETED

When looking at the CPU utilization it is event that MPI is using the most of the available cores getting the closest to the full usage of the cores.

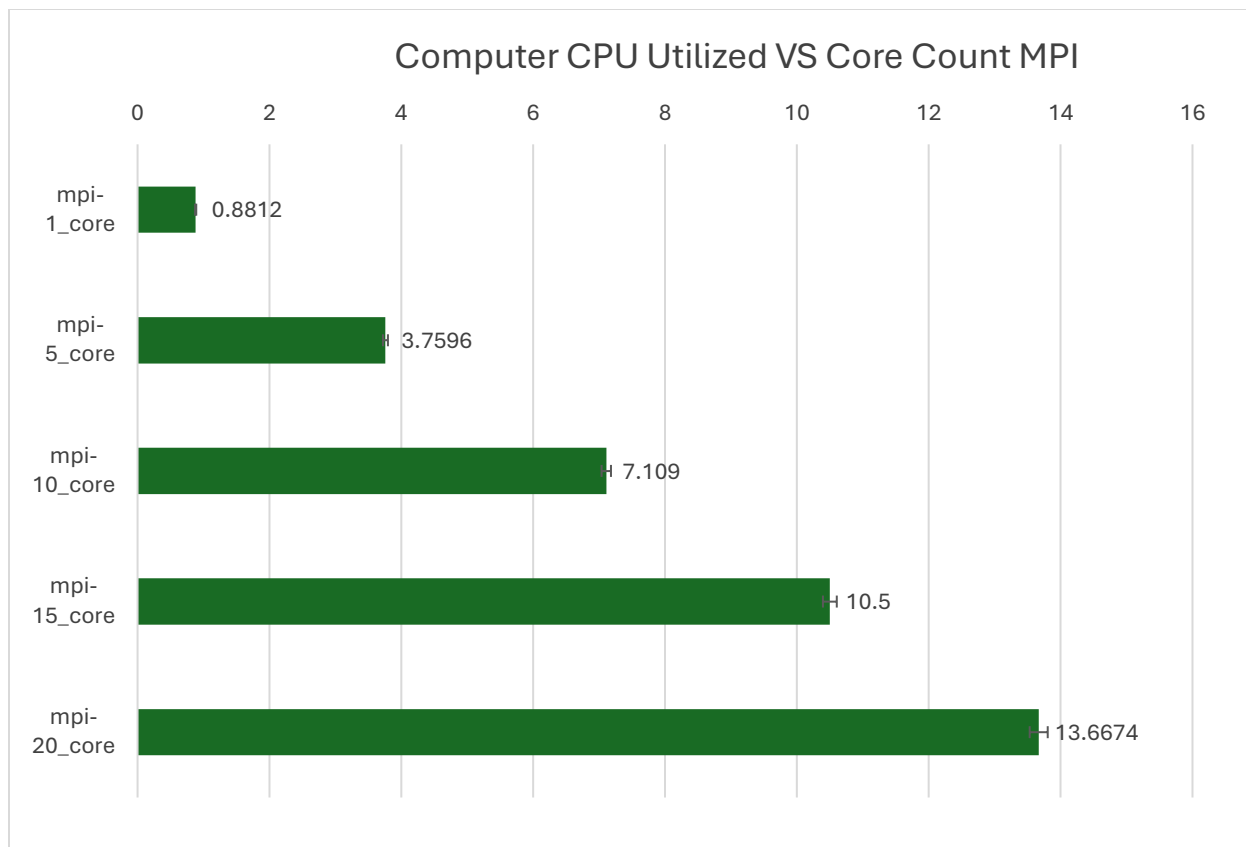


Figure 8: CPU utilization compared to core count

By using perf we were able to get a great amount of information from the programs shown in Appendix A with all our data. Looking at the difference between MPI and the other programs, it was evident that unlike the other programs the number of page faults increased. With it also having a 2x increase to instructions with Pthread having $1.7E+10$ and MPI having $5.7E+10$ with 10 cores. While maintaining the number of branches between the two.

OpenMP-

Following the testing outlined with Pthread we have OpenMP data graphs. As seen below, overall, it ran better than the other two, as it has better implementation with its integrated scheduler. This gives it the advantage over the other two. Thus, it follows the expected curve of with the more cores the lower the time being until a plateau.

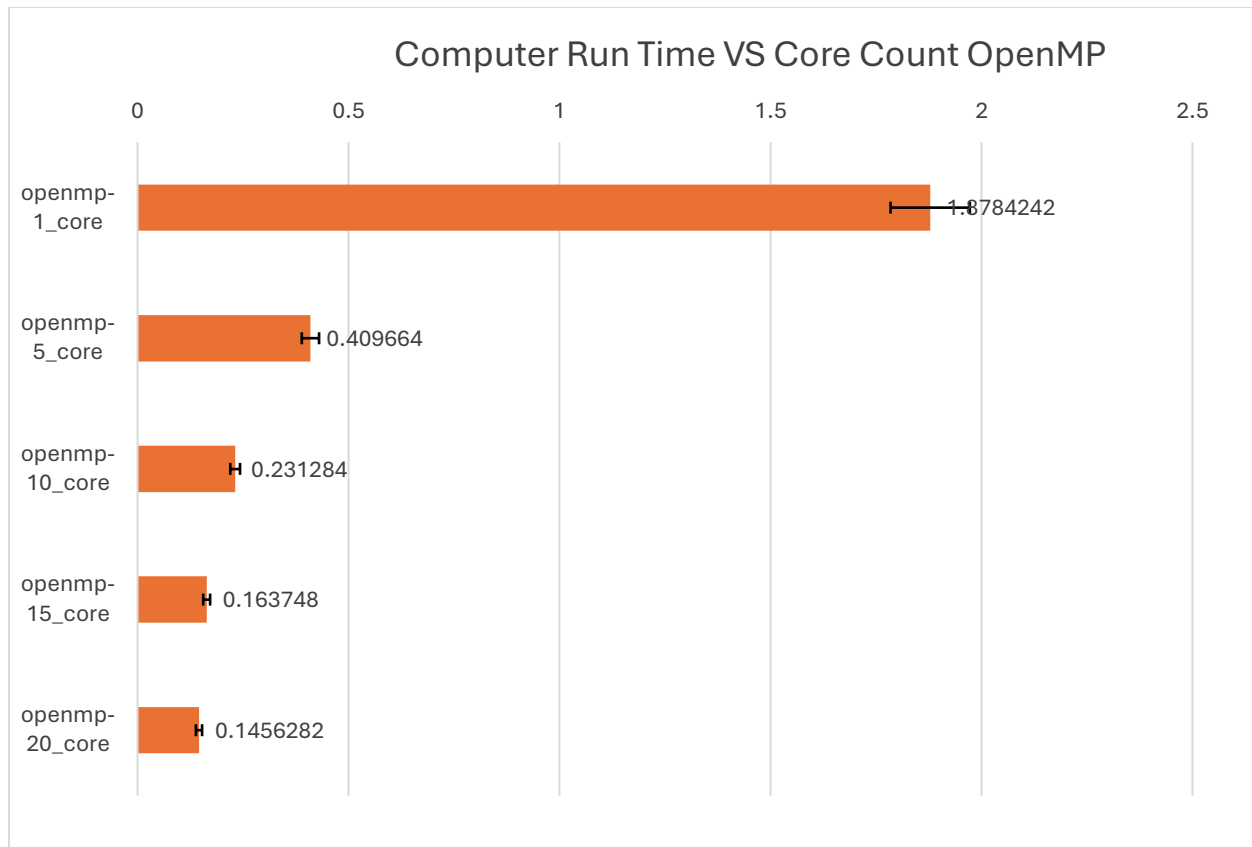


Figure 9: Computer Run Time VS The Core Count Used

The following graph shows the difference between 1 system and 2 system. All other variables were kept the same to limit the differences making sure that the number of cores was the only variable changed. Looking at the data it is noted that the 2 systems took longer than running on one system. This is to be expected as the process must transfer data between the two systems that will never be faster than on chip interconnect speeds. Note that 10 is somewhat close as it was somewhat reasonable to use two systems as the scheduler time was not significant enough to make a difference. Then with 20 cores it took over 3x the time caused by the information transfer. Thus, the system will only benefit if the system was severely limited by the number of cores.

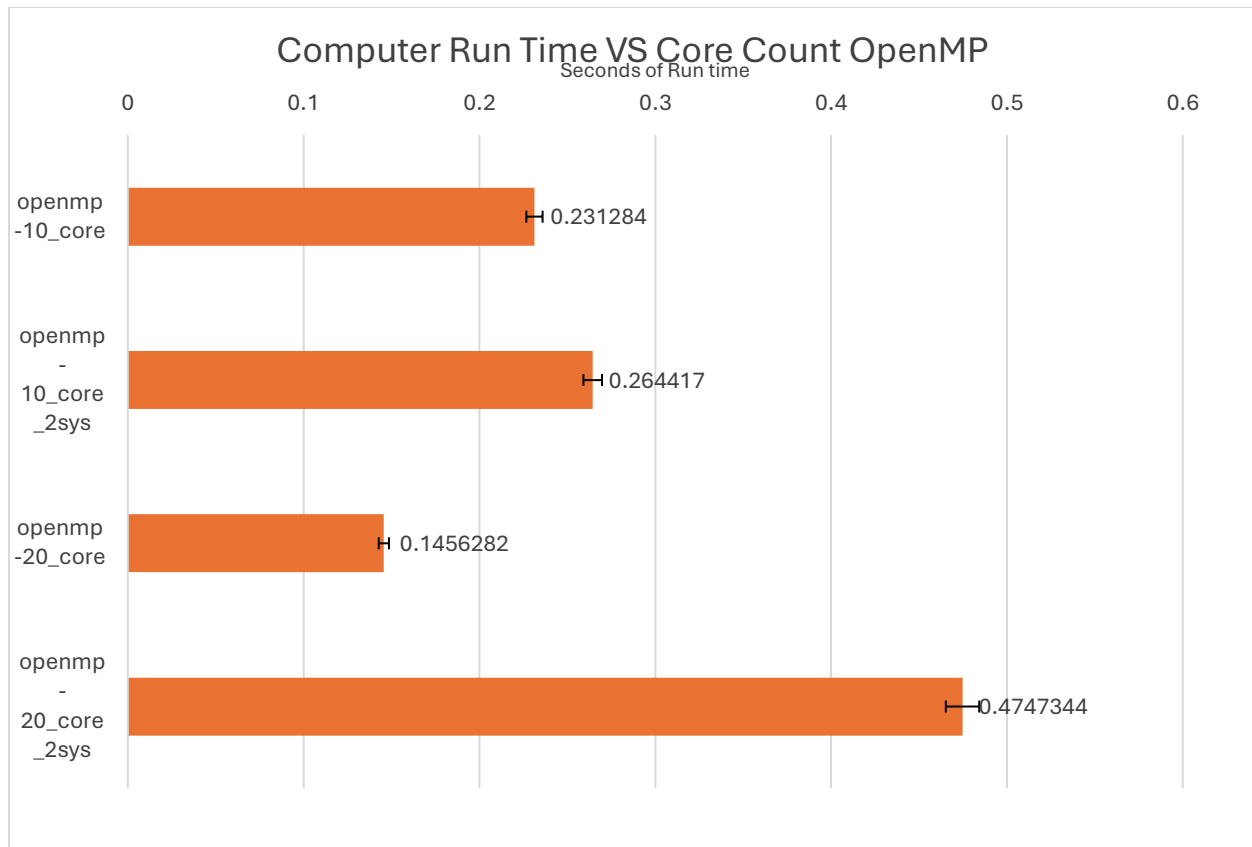


Figure 10: The Compute Run Time Between 1 and 2 systems

The following figure is the different file sizes following the same as Pthread. All these tests were done using 20 cores and the default configuration. Each of the files was built using a python script found in the appendix. The different sizes had the same number of elements in each line, but the number of lines were changed to 1k, 10k 100k and 1M lines. Then each of the 5 tests were run on each of the files. Looking at the graph it is evident that the number of the element's changes in the $O(n^2)$ in time. Overall, it seems to align with expectations but with a comparable time.

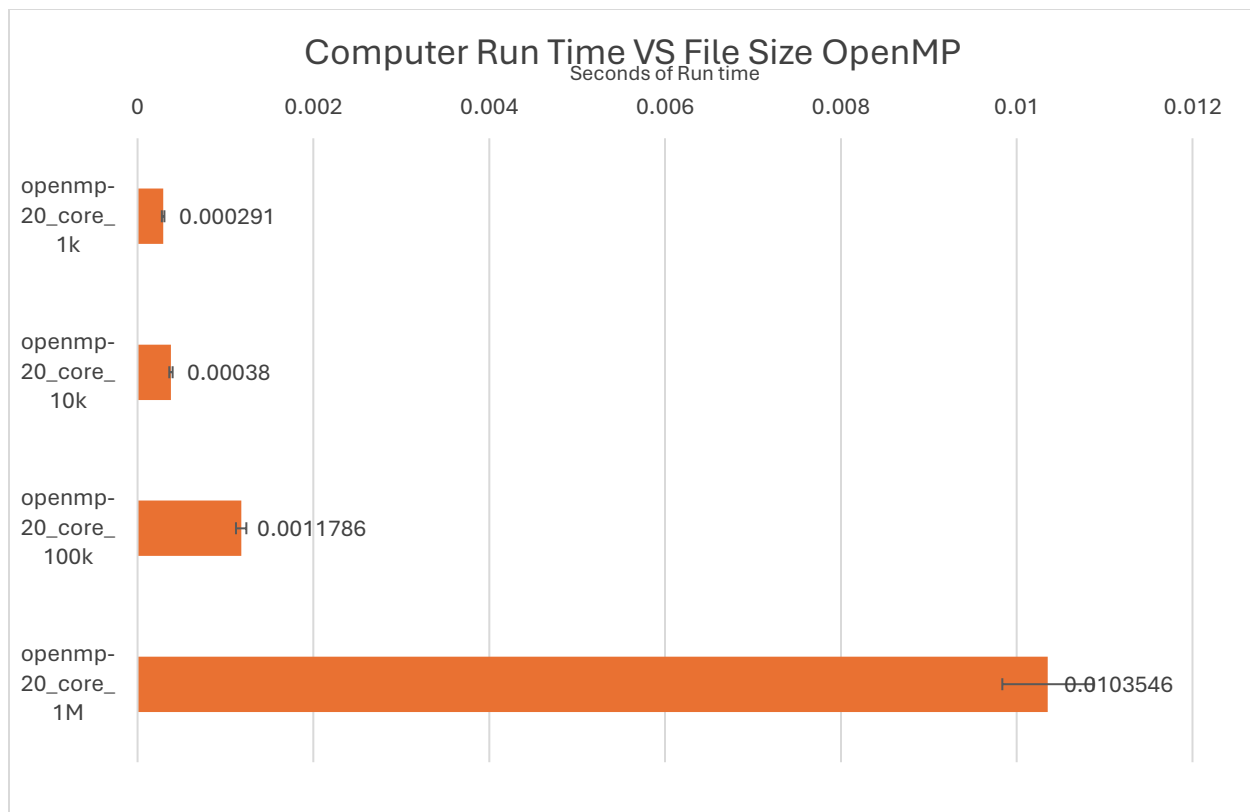


Figure 11: The Computer Run Time VS The Size of the File

Below is the overall memory usage from kstat for pthread. It is evident that the programs seem to use no memory, or less the amount that we would expect. The expected result would be the size of the file to be read or the maximum amount that it could. Evidently, it doesn't show up here. This could be due to many different factors, mostly having to contribute to running on the computer node. Thus, they run almost all using no memory.

21761679	1_core	mole053	1 n	1 c	0.00gb/	7gb	00:00:23	COMPLETED
21761680	5_core	mole054	1 n	5 c	0.00gb/	7gb	00:00:15	COMPLETED
21761681	10_core	mole059	1 n	10 c	0.00gb/	7gb	00:00:15	COMPLETED
21761682	15_core	mole058	1 n	15 c	0.00gb/	7gb	00:00:15	COMPLETED
21761683	20_core	mole055	1 n	20 c	0.00gb/	7gb	00:00:14	COMPLETED
21761684	10_core_2s	mole[053-054]	2 n	10 c	0.00gb/	14gb	00:00:16	COMPLETED
21761685	20_core_2s	mole[053-054]	2 n	20 c	0.00gb/	14gb	00:00:15	COMPLETED
21761686	5_core_1M	mole056	1 n	20 c	0.00gb/	7gb	00:00:00	COMPLETED
21761687	5_core_100	mole057	1 n	20 c	0.00gb/	7gb	00:00:00	COMPLETED
21761688	5_core_10K	mole056	1 n	20 c	0.00gb/	7gb	00:00:00	COMPLETED
21761689	5_core_1K	mole057	1 n	20 c	0.02gb/	7gb	00:00:01	COMPLETED

When looking at the cores utilized it is less than the others, but this is due to efficiency. Using the max with 1 core like the others but after that it is lower usage than the other systems with the same allocated cores correlating to a faster run time.

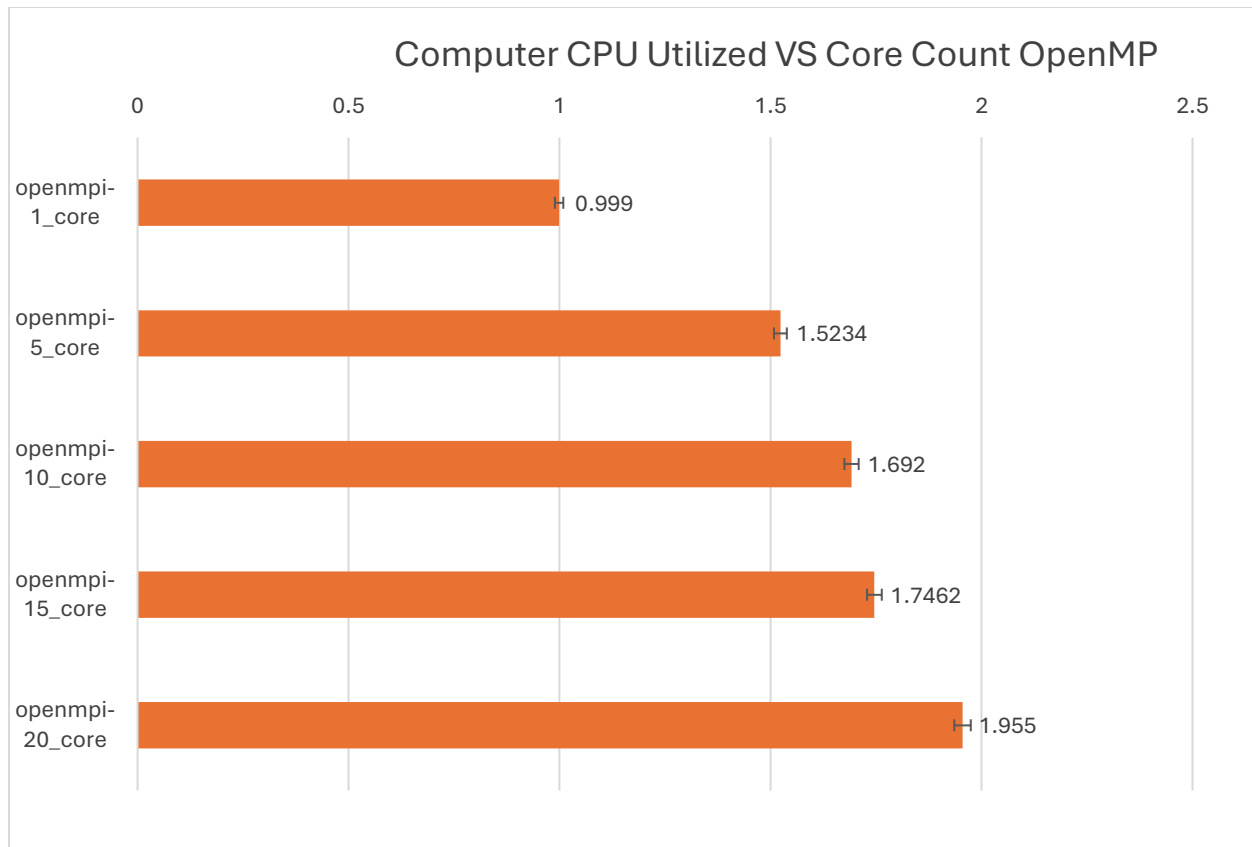


Figure 12: CPU utilization compared to core count

Conclusion:

Our MPI didn't work properly, as we thought it should have printed the same number of lines as OpenMP and Pthread, but it actually printed around double. We investigated it and concluded that it is likely caused by a mismatch between the actual number of lines and how they're distributed among processes, especially during the MPI_Scatter and MPI_Gather operations. We tried a lot of different things in the code but couldn't figure out what we could do to fix the error.

The OpenMP and Pthread implementations were extremely similar to each other, both having 3E+06 number of branch misses with similar branches.

Overall, the other programming threads work great with the systems except for MPI. Threading does help the system complete the task at hand more efficiently and quickly. Thus, allowing you to use all the available hardware to run the program when needed. Comparing the two systems of Pthread and OpenMP they were able to both to bring significant changes with compute times. With OpenMP it was able to win the overall spot, being the fastest of the others. MPI was able to run and had great times even if it ran over double the number of lines that it needed to. It still was difficult to code with and left to be

desired with implementation. In conclusion, multi-threading is a powerful tool to be used in big data and other applications.

References:

Appendix A:

The following is the document that houses all the data. The small snippet of it is as follows.

[Data_CIS520.xlsx](#)

File Name	Thread Times (s).1	Thread Times (s).2	Thread Times (s).3	Thread Times (s).4	Thread Times (s).5	Thread Times Avg (s)
pthread-1_core	2.104007	2.393763	2.040653	1.855762	1.86037	2.050911
pthread-5_core	0.48466	0.503076	0.490398	0.495289	0.551567	0.504998
pthread-10_core	0.359651	0.32636	0.426991	0.4185	0.402621	0.3868246
pthread-15_core	0.362802	0.375489	0.390897	0.38854	0.380103	0.3795662
pthread-20_core	0.39653	0.377781	0.391278	0.3919	0.381079	0.3877136
pthread-20_core_2sys	0.435446	0.452311	0.442236	0.457121	0.462811	0.449985
pthread-10_core_2sys	0.278736	0.27447	0.2783	0.274488	0.273084	0.2758156
pthread-5_core_1K	0.000466	0.000358	0.000323	0.000373	0.000381	0.0003802
pthread-5_core_10k	0.00139	0.001317	0.001206	0.00128	0.001295	0.0012976
pthread-5_core_100k	0.011774	0.011086	0.010499	0.01023	0.009913	0.0107004
pthread-5_core_1M	0.11356	0.105697	0.108891	0.109511	0.109801	0.109492
mpi-1_core	6.669642	6.526187	6.443012	6.436009	6.430612	6.5010924
mpi-5_core	2.170218	2.152979	2.157708	2.062424	2.155909	2.1398476
mpi-10_core	1.428012	1.423592	1.426995	1.476573	1.555594	1.4621532
mpi-15_core	1.299333	1.304102	1.329263	1.303431	1.301778	1.3075814
mpi-20_core	1.189149	1.189423	1.188218	1.189403	1.18902	1.1890426
mpi-20_core_2sys	16.257575	15.897331	15.933152	15.833551	15.790219	15.9423656
mpi-10_core_2sys	16.115503	16.11146	15.709286	15.795529	15.702119	15.8867794

mpi-5_core_1k	0.000969	0.000975	0.00101	0.000992	0.000972	0.0009836
mpi-5_core_10k	0.006801	0.006638	0.006586	0.006727	0.006622	0.0066748
mpi-5_core_100k	0.049765	0.046028	0.049456	0.047012	0.046607	0.0477736
mpi-5_core_1M	0.536535	0.445896	0.444305	0.524822	0.445525	0.4794166
openmp-1_core	1.882377	1.928285	1.92055	1.877408	1.783501	1.8784242
openmp-5_core	0.409971	0.409965	0.409548	0.409671	0.409165	0.409664
openmp-10_core	0.230652	0.233419	0.230724	0.230612	0.231013	0.231284
openmp-15_core	0.164054	0.161352	0.164061	0.163274	0.165999	0.163748
openmp-20_core	0.145389	0.146114	0.145982	0.145581	0.145075	0.1456282
openmp-20_core_2sys	0.423655	0.426863	0.541456	0.46525	0.516448	0.4747344
openmp-10_core_2sys	0.268787	0.266469	0.270455	0.262746	0.253628	0.264417
openmp-5_core_1k	0.000377	0.000257	0.000311	0.000243	0.000267	0.000291
openmp-5_core_10k	0.000379	0.000369	0.000382	0.000381	0.000389	0.00038
openmp-5_core_100k	0.001418	0.001119	0.001133	0.001125	0.001098	0.0011786
openmp-5_core_1M	0.010369	0.010432	0.01016	0.010165	0.010647	0.0103546

Appendix B:

The following is the bash code with each of the programs and well as bash scripts for quick testing.

High Level Bash-

quickstart_build.sh:

```
#!/bin/bash

#This script is to quickly set up the environment for running the programs
# I am lazy to do it manually

chmod +x 3way-pthread/configs.sh
chmod +x 3way-pthread/program_thread.sh
chmod +x 3way-pthread/pthread_dif_files.sh
```

```
#go into the 3way-pthread directory
cd 3way-pthread
#make the program
#make clean
make
#run the configs.sh file
#./configs.sh
#exit
#go back to the main directory
cd ..

chmod +x 3way-mpi/configs.sh
chmod +x 3way-mpi/program_thread.sh
chmod +x 3way-mpi/pthread_dif_files.sh
#go into the 3way-mpi directory
cd 3way-mpi
#make the program
#make clean
make
#run the configs.sh file
#./configs.sh
#exit
#go back to the main directory
cd ..

chmod +x 3way-openmp/configs.sh
chmod +x 3way-openmp/program_thread.sh
chmod +x 3way-openmp/pthread_dif_files.sh

#go into the 3way-openmp directory
cd 3way-openmp
#make the program
#make clean
make
#run the configs.sh file
#./configs.sh
#exit
#go back to the main directory
cd ..
```

quickstart.sh:

```
#!/bin/bash
```

```
#This script is to quickly set up the environment for running the programs
# I am lazy to do it manually
```

```
chmod +x 3way-pthread/configs.sh
chmod +x 3way-pthread/program_thread.sh
chmod +x 3way-pthread/pthread_dif_files.sh
```

```
#go into the 3way-pthread directory
cd 3way-pthread
#make the program
#make clean
make
#run the configs.sh file
./configs.sh
#exit
#go back to the main directory
cd ..
```

```
chmod +x 3way-mpi/configs.sh
chmod +x 3way-mpi/program_thread.sh
chmod +x 3way-mpi/pthread_dif_files.sh
#go into the 3way-mpi directory
cd 3way-mpi
#make the program
#make clean
make
#run the configs.sh file
./configs.sh
#exit
#go back to the main directory
cd ..
```

```
chmod +x 3way-openmp/configs.sh
chmod +x 3way-openmp/program_thread.sh
chmod +x 3way-openmp/pthread_dif_files.sh

#go into the 3way-openmp directory
cd 3way-openmp
#make the program
#make clean
make
#run the configs.sh file
./configs.sh
#exit
#go back to the main directory
```

```
cd ..
```

Pthreads-

configs.sh:

```
#!/bin/sh
sbatch --time=10 --mem=7G --cpus-per-task=1 --threads-per-core=1 --ntasks=1 --
nodes=1 --constraint=moles --job-name='1_core' ./program_thread.sh 1
sbatch --time=5 --mem=7G --cpus-per-task=5 --threads-per-core=1 --ntasks=1 --
nodes=1 --constraint=moles --job-name='5_core' ./program_thread.sh 5
sbatch --time=5 --mem=7G --cpus-per-task=10 --threads-per-core=1 --ntasks=1 --
nodes=1 --constraint=moles --job-name='10_core' ./program_thread.sh 10
sbatch --time=5 --mem=7G --cpus-per-task=15 --threads-per-core=1 --ntasks=1 --
nodes=1 --constraint=moles --job-name='15_core' ./program_thread.sh 15
sbatch --time=5 --mem=7G --cpus-per-task=20 --threads-per-core=1 --ntasks=1 --
nodes=1 --constraint=moles --job-name='20_core' ./program_thread.sh 20

#Run on 2 different system
sbatch --time=5 --mem=7G --cpus-per-task=5 --threads-per-core=1 --ntasks=2 --
nodes=2 --constraint=moles --job-name='10_core_2sys' ./program_thread.sh 10
sbatch --time=5 --mem=7G --cpus-per-task=10 --threads-per-core=1 --ntasks=2 --
nodes=2 --constraint=moles --job-name='20_core_2sys' ./program_thread.sh 20

#This Will run different files for the program
#If you want to run the different files for the program you need to let me know as
the files are not in the repo and are too large to upload
sbatch --time=5 --mem=7G --cpus-per-task=10 --threads-per-core=1 --ntasks=2 --
nodes=1 --constraint=moles --job-name='5_core_1M' ./pthread_dif_files.sh 5
../random_text_1k.txt
sbatch --time=5 --mem=7G --cpus-per-task=10 --threads-per-core=1 --ntasks=2 --
nodes=1 --constraint=moles --job-name='5_core_100K' ./pthread_dif_files.sh 5
../random_text_10k.txt
sbatch --time=5 --mem=7G --cpus-per-task=10 --threads-per-core=1 --ntasks=2 --
nodes=1 --constraint=moles --job-name='5_core_10K' ./pthread_dif_files.sh 5
../random_text_100k.txt
sbatch --time=5 --mem=7G --cpus-per-task=10 --threads-per-core=1 --ntasks=2 --
nodes=1 --constraint=moles --job-name='5_core_1K' ./pthread_dif_files.sh 5
../random_text_1M.txt
```

program_thread.sh:

```
#!/bin/bash
# This script runs the pthread_program with perf stat for a given number of
threads.
echo "I AM $1 Number of cores"
echo "I am using dan/625/wiki_dump.txt for input"
perf stat ./pthread_program ~dan/625/wiki_dump.txt $1
perf stat ./pthread_program ~dan/625/wiki_dump.txt $1
perf stat ./pthread_program ~dan/625/wiki_dump.txt $1
perf stat ./pthread_program ~dan/625/wiki_dump.txt $1
perf stat ./pthread_program ~dan/625/wiki_dump.txt $1
```

pthread_dif_files.sh:

```
#!/bin/bash
echo "I AM $1 Number of cores"
echo "I am $2 for input"
perf stat ./pthread_program $2 $1
perf stat ./pthread_program $2 $1
perf stat ./pthread_program $2 $1
perf stat ./pthread_program $2 $1
perf stat ./pthread_program $2 $1
```

MPI-

configs.sh:

```
#!/bin/sh
sbatch --time=10 --mem=7G --cpus-per-task=1 --threads-per-core=1 --ntasks=1 --
nodes=1 --constraint=moles --job-name='1_core' ./program_thread.sh 1
sbatch --time=5 --mem=7G --cpus-per-task=5 --threads-per-core=1 --ntasks=1 --
nodes=1 --constraint=moles --job-name='5_core' ./program_thread.sh 5
sbatch --time=5 --mem=7G --cpus-per-task=10 --threads-per-core=1 --ntasks=1 --
nodes=1 --constraint=moles --job-name='10_core' ./program_thread.sh 10
sbatch --time=5 --mem=7G --cpus-per-task=15 --threads-per-core=1 --ntasks=1 --
nodes=1 --constraint=moles --job-name='15_core' ./program_thread.sh 15
sbatch --time=5 --mem=7G --cpus-per-task=20 --threads-per-core=1 --ntasks=1 --
nodes=1 --constraint=moles --job-name='20_core' ./program_thread.sh 20

#Run on 2 diffrent system
```

```

sbatch --time=5 --mem=7G --cpus-per-task=5 --threads-per-core=1 --ntasks=2 --
nodes=2 --constraint=moles --job-name='10_core_2sys' ./program_thread.sh 10
sbatch --time=5 --mem=7G --cpus-per-task=10 --threads-per-core=1 --ntasks=2 --
nodes=2 --constraint=moles --job-name='20_core_2sys' ./program_thread.sh 20

#This Will run diffrent files for the program
#If you want to run the diffrent files for the program you need to let me know as
the files are not in the repo and are too large to upload
sbatch --time=5 --mem=7G --cpus-per-task=10 --threads-per-core=1 --ntasks=2 --
nodes=1 --constraint=moles --job-name='5_core_1M' ./pthread_dif_files.sh 5
../random_text_1k.txt
sbatch --time=5 --mem=7G --cpus-per-task=10 --threads-per-core=1 --ntasks=2 --
nodes=1 --constraint=moles --job-name='5_core_100K' ./pthread_dif_files.sh 5
../random_text_10k.txt
sbatch --time=5 --mem=7G --cpus-per-task=10 --threads-per-core=1 --ntasks=2 --
nodes=1 --constraint=moles --job-name='5_core_10K' ./pthread_dif_files.sh 5
../random_text_100k.txt
sbatch --time=5 --mem=7G --cpus-per-task=10 --threads-per-core=1 --ntasks=2 --
nodes=1 --constraint=moles --job-name='5_core_1K' ./pthread_dif_files.sh 5
../random_text_1M.txt

```

program_thread.sh:

```

#!/bin/bash
echo "I AM $1 Number of cores"
echo "I am using dan/625/wiki_dump.txt for input"
perf stat mpirun --oversubscribe -np $1 ./mpi_program ~dan/625/wiki_dump.txt
perf stat mpirun --oversubscribe -np $1 ./mpi_program ~dan/625/wiki_dump.txt
perf stat mpirun --oversubscribe -np $1 ./mpi_program ~dan/625/wiki_dump.txt
perf stat mpirun --oversubscribe -np $1 ./mpi_program ~dan/625/wiki_dump.txt
perf stat mpirun --oversubscribe -np $1 ./mpi_program ~dan/625/wiki_dump.txt

```

pthread_dif_files.sh:

```

#!/bin/bash
echo "I AM $1 Number of cores"
echo "I am using dan/625/wiki_dump.txt for input"
perf stat mpirun --oversubscribe -np $1 ./mpi_program ~dan/625/wiki_dump.txt
perf stat mpirun --oversubscribe -np $1 ./mpi_program ~dan/625/wiki_dump.txt
perf stat mpirun --oversubscribe -np $1 ./mpi_program ~dan/625/wiki_dump.txt
perf stat mpirun --oversubscribe -np $1 ./mpi_program ~dan/625/wiki_dump.txt

```

OpenMP-

configs.sh:

```
#!/bin/sh
sbatch --time=10 --mem=7G --cpus-per-task=1 --threads-per-core=1 --ntasks=1 --
nodes=1 --constraint=moles --job-name='1_core' ./program_thread.sh 1
sbatch --time=5 --mem=7G --cpus-per-task=5 --threads-per-core=1 --ntasks=1 --
nodes=1 --constraint=moles --job-name='5_core' ./program_thread.sh 5
sbatch --time=5 --mem=7G --cpus-per-task=10 --threads-per-core=1 --ntasks=1 --
nodes=1 --constraint=moles --job-name='10_core' ./program_thread.sh 10
sbatch --time=5 --mem=7G --cpus-per-task=15 --threads-per-core=1 --ntasks=1 --
nodes=1 --constraint=moles --job-name='15_core' ./program_thread.sh 15
sbatch --time=5 --mem=7G --cpus-per-task=20 --threads-per-core=1 --ntasks=1 --
nodes=1 --constraint=moles --job-name='20_core' ./program_thread.sh 20

#Run on 2 diffrent system
sbatch --time=5 --mem=7G --cpus-per-task=5 --threads-per-core=1 --ntasks=2 --
nodes=2 --constraint=moles --job-name='10_core_2sys' ./program_thread.sh 10
sbatch --time=5 --mem=7G --cpus-per-task=10 --threads-per-core=1 --ntasks=2 --
nodes=2 --constraint=moles --job-name='20_core_2sys' ./program_thread.sh 20

#This Will run diffrent files for the program
#If you want to run the diffrent files for the program you need to let me know as
the files are not in the repo and are too large to upload
sbatch --time=5 --mem=7G --cpus-per-task=10 --threads-per-core=1 --ntasks=2 --
nodes=1 --constraint=moles --job-name='5_core_1M' ./pthread_dif_files.sh 5
../random_text_1k.txt
sbatch --time=5 --mem=7G --cpus-per-task=10 --threads-per-core=1 --ntasks=2 --
nodes=1 --constraint=moles --job-name='5_core_100K' ./pthread_dif_files.sh 5
../random_text_10k.txt
sbatch --time=5 --mem=7G --cpus-per-task=10 --threads-per-core=1 --ntasks=2 --
nodes=1 --constraint=moles --job-name='5_core_10K' ./pthread_dif_files.sh 5
../random_text_100k.txt
sbatch --time=5 --mem=7G --cpus-per-task=10 --threads-per-core=1 --ntasks=2 --
nodes=1 --constraint=moles --job-name='5_core_1K' ./pthread_dif_files.sh 5
../random_text_1M.txt
```

program_thread.sh:

```
#!/bin/bash
echo "I AM $1 Number of cores"
echo "I am using dan/625/wiki_dump.txt for input"
perf stat ./openmp_program ~dan/625/wiki_dump.txt $1
```

```
perf stat ./openmp_program ~dan/625/wiki_dump.txt $1
perf stat ./openmp_program ~dan/625/wiki_dump.txt $1
perf stat ./openmp_program ~dan/625/wiki_dump.txt $1
perf stat ./openmp_program ~dan/625/wiki_dump.txt $1
```

pthread_dif_files.sh:

```
#!/bin/bash
echo "I AM $1 Number of cores"
echo "I am $2 for input"
perf stat ./openmp_program $2 $1
perf stat ./openmp_program $2 $1
perf stat ./openmp_program $2 $1
perf stat ./openmp_program $2 $1
perf stat ./openmp_program $2 $1
```

Appendix C:

The following is the first 100 lines of MPI, pthread, and OpenMP outputs, respectively. Our MPI implementation did not produce identical results, because MPI_Scatterv and MPI_Gatherv (not to be confused with MPI_Scatter and MPI_Gather), are not guaranteed to return the data in the same order. We tried many different solutions, but in the end, we were not successful in getting the results in the correct order.


```
Line 0 max ASCII: 125
Line 1 max ASCII: 125
Line 2 max ASCII: 125
Line 3 max ASCII: 125
Line 4 max ASCII: 125
Line 5 max ASCII: 125
Line 6 max ASCII: 125
Line 7 max ASCII: 125
Line 8 max ASCII: 125
Line 9 max ASCII: 125
Line 10 max ASCII: 125
Line 11 max ASCII: 124
Line 12 max ASCII: 125
Line 13 max ASCII: 124
Line 14 max ASCII: 125
Line 15 max ASCII: 124
Line 16 max ASCII: 124
Line 17 max ASCII: 121
Line 18 max ASCII: 125
Line 19 max ASCII: 124
Line 20 max ASCII: 125
Line 21 max ASCII: 125
Line 22 max ASCII: 125
Line 23 max ASCII: 124
Line 24 max ASCII: 126
Line 25 max ASCII: 125
Line 26 max ASCII: 125
Line 27 max ASCII: 122
Line 28 max ASCII: 125
Line 29 max ASCII: 125
Line 30 max ASCII: 125
Line 31 max ASCII: 125
Line 32 max ASCII: 125
Line 33 max ASCII: 125
Line 34 max ASCII: 125
Line 35 max ASCII: 125
Line 36 max ASCII: 125
Line 37 max ASCII: 125
Line 38 max ASCII: 125
Line 39 max ASCII: 125
Line 40 max ASCII: 124
Line 41 max ASCII: 124
Line 42 max ASCII: 125
Line 43 max ASCII: 124
Line 44 max ASCII: 125
Line 45 max ASCII: 125
Line 46 max ASCII: 124
Line 47 max ASCII: 125
Line 48 max ASCII: 125
Line 49 max ASCII: 125
Line 50 max ASCII: 125
Line 51 max ASCII: 124
Line 52 max ASCII: 125
Line 53 max ASCII: 125
Line 54 max ASCII: 125
Line 55 max ASCII: 125
Line 56 max ASCII: 125
Line 57 max ASCII: 121
Line 58 max ASCII: 125
Line 59 max ASCII: 125
Line 60 max ASCII: 125
Line 61 max ASCII: 125
Line 62 max ASCII: 125
Line 63 max ASCII: 125
Line 64 max ASCII: 125
Line 65 max ASCII: 125
Line 66 max ASCII: 125
Line 67 max ASCII: 125
Line 68 max ASCII: 125
Line 69 max ASCII: 125
Line 70 max ASCII: 125
Line 71 max ASCII: 125
Line 72 max ASCII: 125
Line 73 max ASCII: 125
Line 74 max ASCII: 125
Line 75 max ASCII: 125
Line 76 max ASCII: 125
Line 77 max ASCII: 125
Line 78 max ASCII: 125
Line 79 max ASCII: 121
Line 80 max ASCII: 125
Line 81 max ASCII: 121
Line 82 max ASCII: 125
Line 83 max ASCII: 125
Line 84 max ASCII: 122
Line 85 max ASCII: 121
Line 86 max ASCII: 125
Line 87 max ASCII: 125
Line 88 max ASCII: 125
Line 89 max ASCII: 124
Line 90 max ASCII: 125
Line 91 max ASCII: 125
Line 92 max ASCII: 125
Line 93 max ASCII: 125
Line 94 max ASCII: 124
Line 95 max ASCII: 125
Line 96 max ASCII: 125
Line 97 max ASCII: 125
Line 98 max ASCII: 124
Line 99 max ASCII: 124
[sagecauley@icr-elymene 3way-mpi]$
```

```
0: 125
1: 125
2: 125
3: 125
4: 125
5: 125
6: 125
7: 125
8: 125
9: 124
10: 125
11: 125
12: 125
13: 126
14: 125
15: 125
16: 125
17: 125
18: 125
19: 125
20: 125
21: 125
22: 125
23: 125
24: 125
25: 124
26: 125
27: 125
28: 125
29: 125
30: 125
31: 125
32: 125
33: 125
34: 125
35: 125
36: 125
37: 125
38: 125
39: 125
40: 125
41: 125
42: 125
43: 125
44: 125
45: 125
46: 125
47: 125
48: 125
49: 125
50: 125
51: 125
52: 122
53: 125
54: 125
55: 125
56: 125
57: 125
58: 125
59: 125
60: 125
61: 125
62: 125
63: 125
64: 125
65: 125
66: 125
67: 125
68: 125
69: 125
70: 124
71: 125
72: 125
73: 125
74: 125
75: 125
76: 125
77: 125
78: 125
79: 125
80: 125
81: 125
82: 125
83: 125
84: 125
85: 125
86: 125
87: 125
88: 125
89: 125
90: 125
91: 125
92: 125
93: 125
94: 125
95: 125
96: 125
97: 122
98: 125
99: 125
[sagecauley@icr-clymene 3way-pthread]$
```

```
0: 125
1: 125
2: 125
3: 125
4: 125
5: 125
6: 125
7: 125
8: 125
9: 124
10: 125
11: 125
12: 125
13: 126
14: 125
15: 125
16: 125
17: 125
18: 125
19: 125
20: 125
21: 125
22: 125
23: 125
24: 125
25: 124
26: 125
27: 125
28: 125
29: 125
30: 125
31: 125
32: 125
33: 125
34: 125
35: 125
36: 125
37: 125
38: 125
39: 125
40: 125
41: 125
42: 125
43: 125
44: 125
45: 125
46: 125
47: 125
48: 125
49: 125
50: 125
51: 125
52: 122
53: 125
54: 125
55: 125
56: 125
57: 125
58: 125
59: 125
60: 125
61: 125
62: 125
63: 125
64: 125
65: 125
66: 125
67: 125
68: 125
69: 125
70: 124
71: 125
72: 125
73: 125
74: 125
75: 125
76: 125
77: 125
78: 125
79: 125
80: 125
81: 125
82: 125
83: 125
84: 125
85: 125
86: 125
87: 125
88: 125
89: 125
90: 125
91: 125
92: 125
93: 125
94: 125
95: 125
96: 125
97: 122
98: 125
99: 125
[sagecauley@icr-clymene 3way-openmp]$
```

Appendix D:

This is the code used to make the different number of lines for testing. This is written in python.

```
import random
import string
# This script generates a text file with random characters
# The file will contain a specified number of lines, each with a specified number
of random characters

#this function make the random text file
def generate_random_text_file(filename, num_lines, num_chars_per_line):
    with open(filename, 'w') as file:
        for line in range(num_lines):
            random_chars = ''.join(random.choices(string.ascii_letters +
string.digits, k=num_chars_per_line))
            file.write(random_chars + '\n')

# The usage
X = 16 # Number of random characters per line
N = 1000 # Number of lines
generate_random_text_file("random_text_1k.txt", N, X)

print("File 'random_text.txt' created successfully!")
```